

Design

Mastermind architecture and reuse plan

The implementation separates the rules of Mastermind from the mechanics shared by a family of turn-based guessing games. The reusable core controls validation, secret creation, history, input/output, and the round loop. The Mastermind package supplies the configuration, scoring algorithm, feedback format, and session setup.

Architecture at a glance

Layer	Primary responsibility	Why the boundary matters
assignment2	Provides the required Driver launch point.	Keeps command-line selection separate from game logic.
assignment2.mastermind	Defines Mastermind-specific rules, defaults, feedback, and session wiring.	A second game can replace this package without rewriting the core loop.
assignment2.core	Provides reusable interfaces, services, value objects, history, and the game engine.	Core code depends on contracts rather than Mastermind classes.

Round flow

1	Setup	MastermindGame selects random or fixed secret generation and constructs the engine.
2	Input	GuessingGameEngine reads a command or guess through UserInterface.
3	Validation	GuessValidator rejects invalid input without consuming an attempt.
4	Evaluation	GuessEvaluator returns Feedback; Mastermind uses black and white peg counts.
5	State	Valid guesses are added to GuessHistory, then the engine decides win, loss, or another turn.

Main conclusion: Phase II should require a new game-specific package and one additional Driver branch, while the core remains unchanged for any game that uses a fixed-length symbol sequence, bounded attempts, and per-guess feedback.

Class and interface diagram

The diagram emphasizes the launch path and the six interface bindings used by the engine. All connectors are routed horizontally and terminate at box boundaries so no connector crosses a type box or another connector.

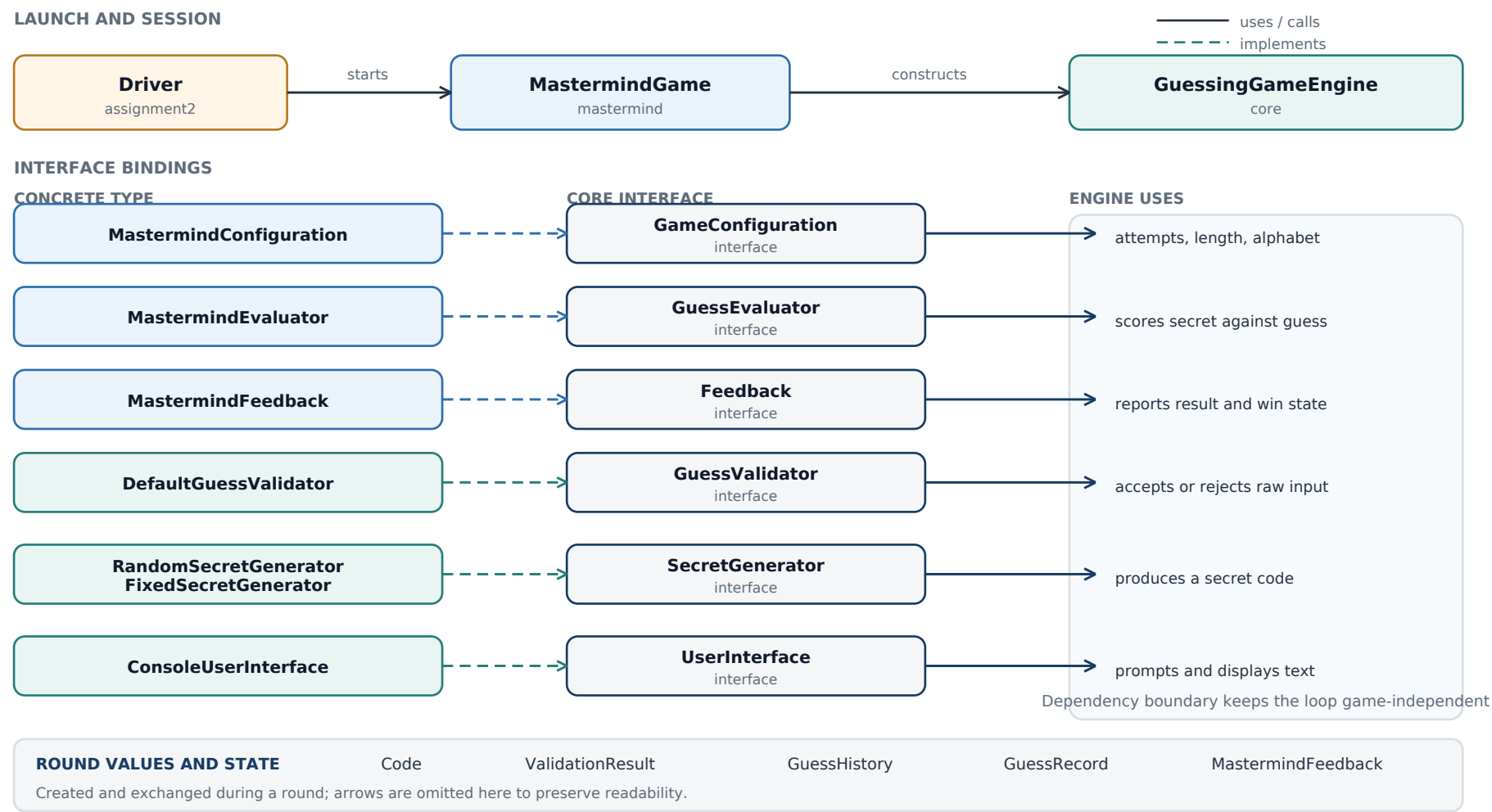


Figure 1. Solid connectors show the launch path or a dependency used by the engine. Dashed connectors show interface implementation. RandomSecretGenerator and FixedSecretGenerator are separate classes that implement the same SecretGenerator contract.

Core responsibility matrix

Each core type has one primary responsibility. The reuse column states the condition under which the type can carry forward to the next game.

Type	Kind	Responsibility	Reuse boundary
Code	class	Immutable ordered sequence used for both secrets and guesses.	Reusable for any fixed-length symbol game.
GameConfiguration	interface	Exposes attempt limit, code length, and legal alphabet.	Reusable while the next game fits these three configuration dimensions.
SecretGenerator	interface	Produces one secret for a round.	Allows random, fixed, or future sources without changing the engine.
RandomSecretGenerator	class	Draws configured symbols uniformly with repetition.	Reusable when repeated symbols are legal.
FixedSecretGenerator	class	Returns a predetermined Code for deterministic testing.	Reusable for repeatable tests in any compatible game.
GuessValidator	interface	Converts raw input into a valid Code or an error.	Lets games substitute specialized validation.
DefaultGuessValidator	class	Checks nonempty input, exact length, and alphabet membership.	Reusable when those checks fully define valid input.
ValidationResult	class	Carries either a parsed Code or a human-readable rejection reason.	Reusable with any GuessValidator implementation.
Feedback	interface	Reports whether a guess wins and supplies display text.	Reusable contract; each game provides its own result type.
GuessEvaluator	interface	Scores a guess against the secret and returns Feedback.	Reusable contract; scoring rules remain game-specific.
GuessHistory	class	Stores valid guesses and feedback in attempt order.	Reusable for games that expose a history command.
GuessRecord	class	Represents one numbered guess and its feedback.	Reusable wherever GuessHistory is used.
UserInterface	interface	Abstracts display and prompt operations.	Supports console or test doubles without changing the engine.
ConsoleUserInterface	class	Implements UserInterface with System.in and System.out.	Reusable for text-based console games.
GuessingGameEngine	class	Runs one round, handles HISTORY and test-mode REVEAL, records valid guesses, and reports outcomes.	Reusable while the next game follows the same round model.

Game-specific responsibilities and extension plan

Type	Responsibility	Classification
Driver	Parses the game name and optional test flag, then launches the selected game.	Required launch adapter
MastermindGame	Displays the greeting, chooses the secret generator, assembles dependencies, runs rounds, and handles replay.	Mastermind-specific session
MastermindConfiguration	Provides defaults of 12 attempts, 4 pegs, and colors B, G, O, P, R, Y; also validates custom values.	Mastermind-specific implementation
MastermindEvaluator	Uses a two-pass algorithm: exact matches first, then remaining color matches with no double counting.	Mastermind-specific rule
MastermindFeedback	Stores black and white peg counts, formats nB_mW, and identifies an all-black win.	Mastermind-specific result

Phase II reuse plan

Reuse unchanged	Replace or extend	Trigger for revisiting the core
Code; configuration and evaluator contracts; validation; secret generators; history; UI; game engine.	A new game package containing configuration, evaluator, feedback, and a session class; one Driver branch.	Variable-length answers, a different turn model, or feedback that cannot be represented by a win flag plus display text.

Key design decisions

- Secret generation, validation, and scoring are separate contracts because each changes for a different reason.
- Fixed and random secrets are separate SecretGenerator implementations, so testing does not add branches to the engine.
- Code is immutable, preventing a recorded guess or secret from changing after creation.
- Invalid input returns ValidationResult.failure instead of throwing an exception because invalid guesses are expected user behavior.
- No general command framework was added for HISTORY and REVEAL; two direct checks are clearer at the current scope.

Known boundary: the reuse plan deliberately targets fixed-length, symbol-based guessing games. It avoids predicting a more general framework before Phase II reveals the next game's requirements.